



Discord Music Bot

Especificaciones Técnicas Detalladas

Documento Completo de Desarrollo e Implementación
Creado: 19 de April de 2026
Versión: 2.0 - Expandida

Atributo	Valor	Detalles
Nombre del Bot	AISAK	Advanced AI-powered Sound and Knowledge system
Plataforma	Discord	Aplicación para servidor de voz y chat
Hosting	Hugging Face Spaces	Servicio gratuito de cloud con Docker support
Gestión de Uptime	UptimeRobot	Monitoreo gratuito cada 5 minutos
Lenguaje Principal	Python 3.9+	Lenguaje moderno y versátil para bots
Framework Discord	discord.py 2.0+	Librería oficial para crear bots Discord
Extractor de Audio	yt-dlp	Fork mantenido de youtube-dl, 1000+ sitios soportados
Procesador de Audio	FFmpeg	Codec de audio de clase mundial
Servidor HTTP	Flask	Micro-framework para mantener conexión viva

1. DESCRIPCIÓN GENERAL Y VISIÓN DEL PROYECTO

AISAK es un bot de Discord de última generación diseñado para proporcionar experiencias de reproducción de música de calidad profesional en servidores de Discord. A diferencia de otros bots de música que dependen de APIs restrictivas o servicios de terceros limitados, AISAK implementa una arquitectura escalable y resiliente que puede reproducir música desde prácticamente cualquier fuente de internet. El sistema está construido sobre el siguiente paradigma: **cualquier canción, en cualquier momento, desde cualquier lugar**. Esto significa que el usuario puede solicitar una canción por nombre, artista, URL de Spotify, enlace de YouTube, o incluso una búsqueda genérica, y el bot la encontrará y reproducirá instantáneamente en el canal de voz. El bot será alojado en **Hugging Face Spaces**, una plataforma gratuita y escalable que soporta Docker, permitiendo total control sobre el entorno de ejecución. Para garantizar disponibilidad 24/7 sin intervención manual, se implementará **UptimeRobot**, que realizará pings periódicos al servidor Flask del bot, evitando que entre en estado de hibernación.

1.1 Objetivos Clave del Proyecto

- ✓ **Reproducción Universal de Música:** Soportar reproducción desde YouTube, Spotify, Deezer, SoundCloud, Apple Music, Tidal, Amazon Music, Bandcamp, y cualquier URL de archivo de audio directo.
- ✓ **Búsqueda Inteligente:** Implementar búsqueda por nombre de canción, artista, álbum, URL o incluso consultas genéricas que el bot interprete para encontrar la mejor coincidencia.
- ✓ **Gestión de Cola Avanzada:** Sistema de cola de reproducción con soporte para reordenamiento, eliminación selectiva, loops, shuffle, y visualización en tiempo real.
- ✓ **Disponibilidad Garantizada:** Mantener el bot en línea 24/7 sin intervención manual mediante UptimeRobot y arquitectura resiliente en Hugging Face Spaces.
- ✓ **Experiencia de Usuario Intuitiva:** Proporcionar comandos claros, respuestas informativas, y feedback visual en embeds de Discord.
- ✓ **Escalabilidad Multi-Servidor:** El bot debe soportar simultáneamente múltiples servidores Discord con colas independientes y sin conflictos de recursos.

1.2 Diferenciadores de AISAK

- **Arquitectura Modular:** Uso de Cogs de discord.py permite agregar/remover funcionalidades sin afectar la estabilidad.
- **Manejo Robusto de Errores:** Cada comando incluye validaciones exhaustivas y manejo de excepciones graceful.
- **Seguridad de Credenciales:** Todas las credenciales sensibles se almacenan en variables de entorno de HF Spaces, nunca en el código fuente.
- **Logging Detallado:** Sistema de logging que registra eventos importantes sin comprometer privacidad o seguridad.
- **Integraciones Múltiples:** Soporte simultáneo para múltiples APIs de música, con fallback automático si una API falla.

2. ARQUITECTURA TÉCNICA DETALLADA

2.1 Componentes Principales del Stack Tecnológico

La arquitectura de AISAK se construye sobre una pila de tecnologías cuidadosamente seleccionadas, cada una elegida por su robustez, mantenibilidad y compatibilidad:

Componente	Tecnología	Versión	Propósito
Runtime Base	Python	3.9+	Lenguaje de programación principal del bot
API de Discord	discord.py	2.0+	Librería oficial para interactuar con Discord API v10
Extracción de Audio	yt-dlp	latest	Descargar y extraer metadatos de 1000+ sitios de video/música
Procesamiento Audio	FFmpeg	latest	Codificar, decodificar y transmitir audio en tiempo real
Servidor HTTP	Flask	2.0+	Micro-framework para endpoint que mantiene vivo el bot
Gestión de Env	python-dotenv	1.0+	Cargar variables de entorno desde archivo .env
Cliente HTTP	requests	2.31+	Realizar peticiones HTTP a APIs externas
Async HTTP	aiohttp	3.9+	Cliente HTTP asíncronico para no bloquear el bot
Hosting	HF Spaces	-	Plataforma cloud con Docker, GPU/CPU flexible
Uptime Mgmt	UptimeRobot	-	Servicio de monitoreo que pinga al bot cada 5 minutos

2.2 Diagrama de Flujo de Arquitectura

El flujo completo de ejecución cuando un usuario solicita una canción funciona de la siguiente manera:

PASO 1 - Entrada del Usuario en Discord:

El usuario escribe un comando en Discord, por ejemplo: */play Shape of You Ed Sheeran*
Discord envía este evento a través de su API al bot que está escuchando en la websocket.

PASO 2 - Procesamiento del Comando:

El decorador `@bot.command()` de discord.py intercepta el comando. El bot valida que:

- El usuario esté en un canal de voz
- El bot tenga permisos para conectarse y hablar en ese canal
- El parámetro (canción) no esté vacío

PASO 3 - Búsqueda con yt-dlp:

El bot invoca yt-dlp con la query del usuario. yt-dlp automáticamente:

- Detecta si es una URL o una búsqueda textual
- Si es búsqueda, realiza búsqueda en YouTube (o sitio configurado)

- Extrae metadatos: título, duración, artista, thumbnails
- Obtiene la URL del stream de audio

PASO 4 - Conexión al Canal de Voz:

El bot se conecta al canal de voz del usuario usando discord.py VoiceClient. Si ya está conectado, reutiliza la conexión existente.

PASO 5 - Procesamiento de Audio con FFmpeg:

discord.FFmpegPCMAudio decodifica el stream de audio obtenido por yt-dlp. FFmpeg convierte cualquier formato (MP4, WebM, MP3, etc.) a PCM compatible con Discord.

PASO 6 - Transmisión al Canal de Voz:

El audio PCM se transmite en tiempo real al canal de voz Discord usando el codec Opus. Los usuarios en el canal escuchan la música simultáneamente.

PASO 7 - Gestión de Cola:

Mientras se reproduce una canción, el bot mantiene una cola de próximas canciones. Cuando una canción termina, automáticamente comienza la siguiente de la cola.

PASO 8 - Monitoreo con UptimeRobot:

UptimeRobot realiza un HTTP GET a <https://tu-space.hf.space/> cada 5 minutos. Flask responde "AISAK Bot está en línea", lo que evita que HF Spaces hiberne el contenedor.

2.3 Estructura de Directorios y Organización del Código

La estructura del proyecto está diseñada siguiendo mejores prácticas de Python y discord.py, permitiendo escalabilidad y mantenibilidad:

```
aisak-bot/ (raíz del proyecto)
■ ■ ■ main.py → Punto de entrada principal. Instancia el bot, carga cogs, inicia Flask
■ ■ ■ config.py → Constantes y configuración (prefijos, timeouts, límites)
■ ■ ■ keep_alive.py → Servidor Flask que responde a UptimeRobot para evitar hibernación
■ ■ ■ requirements.txt → Dependencias de Python con versiones pinned
■ ■ ■ Dockerfile → Especificación de imagen Docker para HF Spaces
■ ■ ■ .env → Variables de entorno (DISCORD_TOKEN, etc.). NO commitar a git
■ ■ ■ .gitignore → Archivos a ignorar en git (.env, __pycache__, .venv)
■
■ ■ ■ cogs/ (Cogs = extensiones/módulos del bot)
■ ■ ■ ■ ■ music.py → Lógica principal de reproducción (play, pause, resume, skip)
■ ■ ■ ■ ■ queue.py → Gestión de cola (add, remove, clear, shuffle, reorder)
■ ■ ■ ■ ■ controls.py → Controles avanzados (volume, pitch, speed, filters)
■ ■ ■ ■ ■ search.py → Búsqueda avanzada y sugerencias
■ ■ ■ ■ ■ help.py → Sistema de ayuda con embeds interactivos
■
■ ■ ■ utils/ (Utilidades compartidas)
■ ■ ■ ■ ■ audio_handler.py → Funciones para extraer y procesar audio
■ ■ ■ ■ ■ validators.py → Validaciones de entrada (URLs, queries)
■ ■ ■ ■ ■ formatters.py → Formateo de respuestas (embeds, mensajes)
■ ■ ■ ■ ■ logger.py → Sistema de logging estructurado
■ ■ ■ ■ ■ errors.py → Excepciones personalizadas del bot
■
■ ■ ■ data/ (Datos persistentes, si se requieren)
■ ■ ■ ■ ■ cache/ → Cache de búsquedas recientes
```

■■■ logs/ → Archivos de log (opcional)

3. DEPENDENCIAS, INSTALACIÓN Y CONFIGURACIÓN

3.1 Archivo requirements.txt Completo con Explicaciones

Cada línea en requirements.txt representa una dependencia Python. La versión está "pinned" (fija) para garantizar reproducibilidad y evitar breaking changes:

```
# Core Discord Bot Framework
discord.py==2.3.2 # Librería oficial Discord API, necesaria para interactuar con Discord

# Audio Extraction & Processing
yt-dlp==2024.01.01 # Fork mantenido de youtube-dl, extrae audio de 1000+ sitios
pydub==0.25.1 # Librería para manipulación de audio (opcional, para filtros)

# Environment & Configuration
python-dotenv==1.0.0 # Lee variables de .env para credenciales seguras

# Web Server & HTTP
Flask==3.0.0 # Micro-framework para mantener bot vivo con endpoint
requests==2.31.0 # Cliente HTTP síncrono (para requests simples)
aiohttp==3.9.1 # Cliente HTTP asíncrono (no bloquea el bot)

# Optional: Logging & Monitoring
python-json-logger==2.0.7 # Logs en formato JSON para mejor análisis

# Optional: Spotify Integration
spotipy==2.22.1 # API oficial de Spotify para búsqueda mejorada
```

3.2 Instalación de Dependencias del Sistema (OS)

Además de dependencias Python, el bot requiere herramientas a nivel sistema operativo. En Hugging Face Spaces (Linux/Ubuntu), estas se instalan en el Dockerfile:

```
FFmpeg: Procesador de audio profesional. Necesario para decodificar video/audio
$ apt-get install ffmpeg
Instalación: libffmpeg para transcoding

Libopus: Codec de audio de Discord. Necesario para transmitir PCM a Discord
$ apt-get install libopus0

Libsodium: Librería de criptografía usada por discord.py para encriptación
$ apt-get install libsodium23

Git: Opcional, para clonar repositorios durante build
$ apt-get install git

Build Tools: Opcional, para compilar extensiones C si es necesario
$ apt-get install build-essential
```

4. COMANDOS PREDETERMINADOS - ESPECIFICACIONES DETALLADAS

AISAK implementará un conjunto robusto de comandos diseñados para ser intuitivos y poderosos. Todos los comandos usan el prefijo "/" (slash commands de Discord) para mejor UX:

Comando	Parámetros	Descripción Funcional
/play	<canción/URL/ID>	Busca y reproduce una canción. Soporta: nombre, URL YouTube, URL Spotify, ID Spotify, búsqueda
/pause	-	Pausa la reproducción actual sin perder posición. Permite reanudar exactamente donde se pausó
/resume	-	Reanuda la reproducción desde donde fue pausada. Error si no hay canción pausada
/skip	[número]	Salta a la siguiente canción. Soporta /skip 3 para saltar 3 canciones
/queue	[página]	Muestra las próximas 10 canciones en cola. Soporta /queue 2 para página 2
/nowplaying	-	Embed detallado de la canción actual: título, artista, duración, progreso, thumbnail
/remove	<posición>	Elimina una canción específica de la cola por número de posición
/clear	-	Limpia toda la cola. Requiere confirmación para evitar accidentes
/shuffle	-	Mezcla aleatoriamente el orden de las canciones en la cola
/repeat	[modo]	Ciclo de repetición: off → canción actual → cola completa
/volume	<0-100>	Ajusta volumen del bot de 0% (silencio) a 100% (máximo)
/stop	-	Detiene reproducción y desconecta el bot del canal de voz
/search	<término>	Busca canciones sin reproducir. Muestra top 5 resultados con números para seleccionar
/lyrics	-	Obtiene y muestra letras de la canción actual (requiere API externa)
/help	[comando]	Muestra lista de comandos o ayuda detallada de comando específico

4.1 Flujo de Ejecución de Comandos - Ejemplo: /play

ENTRADA: Usuario: "/play Shape of You"

1. VALIDACIÓN INICIAL:

- ¿El usuario está en canal de voz? Si no → Error: "Debes estar en un canal de voz"
- ¿El parámetro está vacío? Si sí → Error: "Debes especificar una canción"
- ¿El bot tiene permisos? Validar permiso CONNECT y SPEAK

2. BUSCAR CANCIÓN:

- Ejecutar: yt-dlp con query "Shape of You"
- yt-dlp automáticamente busca en YouTube y retorna:

```
{  
  "title": "Ed Sheeran - Shape of You (Official Video)",  
  "duration": 233,  
  "uploader": "Ed Sheeran",  
  "url": "https://www.youtube.com/watch?v=...",  
  "thumbnail": "https://...",  
  "formats": [...] # Lista de formatos disponibles  
}
```

3. CONECTAR A CANAL DE VOZ:

- Si bot no está conectado: `await channel.connect()`
- Si ya está conectado: reutilizar conexión existente

4. REPRODUCIR AUDIO:

- Obtener URL de stream de audio (formato bestaudio)
- Crear `FFmpegPCMAudio` con opciones específicas
- Enviar PCM stream a Discord voice gateway

5. ACTUALIZAR ESTADO:

- Guardar en queue local: `{title, duration, artist, url, ...}`
- Enviar embed a canal: "■ ■ ■ Ahora reproduciendo: Shape of You"
- Actualizar presencia del bot: "Listening to Shape of You"

6. MONITOREAR REPRODUCCIÓN:

- Esperar evento "after" de `discord.py`
- Cuando termina, reproducir siguiente canción de cola

5. IMPLEMENTACIÓN DE CÓDIGO - ESTRUCTURA Y EJEMPLOS

5.1 main.py - Entrada Principal Detallada

El archivo main.py es el punto de entrada del bot. Realiza las siguientes operaciones críticas: 1. Cargar variables de entorno de .env 2. Configurar intents de Discord (qué eventos escuchar) 3. Instanciar el cliente del bot 4. Registrar event handlers (@bot.event) 5. Cargar extensiones (cogs) 6. Iniciar servidor Flask 7. Conectar a Discord

```
# main.py - Punto de entrada principal del bot AISAK
import discord
from discord.ext import commands
import os
from dotenv import load_dotenv
from keep_alive import keep_alive
import logging

# Configurar logging
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

# Cargar variables de entorno desde archivo .env
load_dotenv()
TOKEN = os.getenv('DISCORD_TOKEN')
if not TOKEN:
    logger.error("DISCORD_TOKEN no encontrado en .env")
    exit(1)

# Configurar intents (permisos para qué eventos recibir)
intents = discord.Intents.default()
intents.message_content = True
# Permite leer contenido de mensajes
intents.voice_states = True
# Permite detectar cambios en canales de voz
intents.guild_messages = True
# Permite recibir mensajes del servidor

# Instanciar el bot con prefijo "/" (slash commands)
bot = commands.Bot(command_prefix='/', intents=intents)

# Evento: Bot conectado y listo
@bot.event
async def on_ready():
    logger.info(f"✓ {bot.user} conectado a Discord")
    await bot.change_presence(activity=discord.Activity(type=discord.ActivityType.listening, name="/help para más comandos | Música en vivo 🎵" ))

# Sincronizar slash commands con Discord
try:
    synced = await bot.tree.sync()
    logger.info(f"✓ {len(synced)} comandos sincronizados")
except Exception as e:
    logger.error(f"Error sincronizando comandos: {e}")

# Evento: Error global en comandos
@bot.event
async def on_command_error(ctx, error):
    logger.error(f"Error en comando {ctx.command}: {error}")
    await ctx.send(f"❌ Error: {str(error)[:100]}")

# Evento: Cambio de estado de usuario (conecta/desconecta de voz)
@bot.event
async def on_voice_state_update(member, before, after):
    # Si el bot es el único en el canal, desconectarse
    if member == bot.user:
        return

    # Lógica adicional si es necesario

# Cargar todas las extensiones (cogs) del directorio cogs/
async def load_cogs():
    for filename in os.listdir('./cogs'):
        if filename.endswith('.py'):
            try:
                await bot.load_extension(f'cogs.{filename[:-3]}')
                logger.info(f"✓ Cog cargado: {filename}")
            except Exception as e:
                logger.error(f"❌ Error cargando {filename}: {e}")

# Función asincrónica para iniciar el bot
async def main():
    async with bot:
        # Cargar extensiones antes de conectar
        await load_cogs()

        # Iniciar servidor Flask (mantiene bot vivo)
        keep_alive()
        logger.info("✓ Servidor Flask iniciado en puerto 8080")

        # Conectar a Discord
        await bot.start(TOKEN)

# Punto de entrada
if __name__ == '__main__':
    import asyncio
    try:
        asyncio.run(main())
    except KeyboardInterrupt:
        logger.info("Bot detenido por usuario")
    except Exception as e:
        logger.critical(f"Error crítico: {e}")
```

5.2 keep_alive.py - Servidor Flask para Uptime

Este archivo es crítico para mantener el bot vivo. Implementa un servidor Flask simple que responde a peticiones HTTP. UptimeRobot hará un GET a este endpoint cada 5 minutos. Si el servidor responde, HF Spaces sabe que el contenedor está activo y no lo hiberna.

```
# keep_alive.py - Servidor Flask para evitar hibernación en HF Spaces
from flask import Flask
import jsonify
from threading import Thread
import logging
logger = logging.getLogger(__name__)

app = Flask(__name__)

@app.route('/')
def home():
    ''' Endpoint raíz que responde a UptimeRobot
    UptimeRobot hace GET a esta ruta cada 5 minutos
    Una respuesta HTTP 200 indica que el bot está vivo '''
    return jsonify({'status': 'online', 'message': 'AISAK Bot está en línea', 'version': '1.0' }), 200

@app.route('/health')
def health():
    '''Endpoint de health check adicional'''
    return jsonify({'health': 'ok'}), 200

@app.route('/status')
def status():
    '''Endpoint que retorna información del bot'''
    return jsonify({'bot': 'AISAK', 'status': 'active', 'uptime': 'tracking enabled' }), 200
```

```
keep_alive(): ''' Inicia el servidor Flask en un thread daemon Esto permite que el bot
Discord y Flask corran simultáneamente ''' t = Thread(target=run_flask) t.daemon = True #
Thread daemon cierra cuando el programa principal termina t.start() logger.info("✓ Servidor
Flask iniciado (thread daemon)") def run_flask(): '''Inicia el servidor Flask''' try: # Host
0.0.0.0 permite conexiones desde cualquier IP # Puerto 8080 es estándar en HF Spaces
app.run(host='0.0.0.0', port=8080, debug=False) except Exception as e: logger.error(f"Error
en servidor Flask: {e}")
```

6. INTEGRACIÓN DE FUENTES DE MÚSICA - ARQUITECTURA DETALLADA

6.1 yt-dlp: El Extractor Universal de Audio

yt-dlp es un fork activamente mantenido de youtube-dl que soporta descargar de más de 1000 sitios web. Es la herramienta más versátil y robusta para extraer audio en la actualidad. El nombre viene de "YouTube-DL Plus", indicando que es una versión mejorada del original. **Capacidades principales:**

- Extraer metadatos sin descargar el archivo (título, duración, artista)
- Obtener URLs de stream de audio/video de múltiples calidades
- Soportar playlists completas
- Manejar cookies para contenido restringido geográficamente
- Fallback automático si un sitio cambia su estructura
- Actualización automática de extractores

Sitios soportados por yt-dlp:

YouTube	Playlists, canales, videos
Spotify	Canciones, playlists, álbumes
Deezer	Canciones, playlists, álbumes
SoundCloud	Tracks, playlists, usuarios
Apple Music	Canciones, playlists, álbumes
Tidal	Canciones, playlists, álbumes
Amazon Music	Canciones, playlists
Bandcamp	Tracks, álbumes
Twitch	VODs y streams en vivo
Dailymotion	Videos
Vimeo	Videos
Mixcloud	Podcasts y DJ mixes
Reddit	Contenido de video
URLs directas	MP3, WAV, FLAC, etc.

6.2 Ejemplo: Extracción de Audio con yt-dlp

```
import yt_dlp async def extract_audio_info(query_or_url): ''' Extrae información de audio sin
descargar el archivo completo Retorna metadatos y URL de stream ''' ydl_opts = { 'format':
'bestaudio/best', # Mejor audio disponible 'quiet': True, 'no_warnings': True,
'extract_flat': False, 'socket_timeout': 30 } try: with yt_dlp.YoutubeDL(ydl_opts) as ydl: #
Si es URL, extraer directamente # Si es búsqueda, yt-dlp automáticamente busca en YouTube
info = ydl.extract_info(query_or_url, download=False) return { 'title': info.get('title'),
'duration': info.get('duration'), 'uploader': info.get('uploader'), 'thumbnail':
info.get('thumbnail'), 'url': info.get('url'), # URL del stream de audio 'formats':
info.get('formats', []) } except Exception as e: raise Exception(f"Error extrayendo audio:
{str(e)}") # Uso en comando /play async def play_command(query): try: audio_info = await
extract_audio_info(query) # Pasar audio_info a FFmpeg para reproducir return audio_info
except Exception as e: print(f"Error: {e}")
```

6.3 FFmpeg: Procesamiento y Codificación de Audio

FFmpeg es la herramienta de software más poderosa para procesamiento multimedia. En el contexto de AISAK, FFmpeg realiza dos funciones críticas: **1. Decodificación:** Convierte cualquier formato de audio (MP4, WebM, MP3, FLAC, etc.) al formato PCM (Pulse Code Modulation) que Discord espera.

2. Codificación con Opus: Codifica el PCM al codec Opus que Discord utiliza nativamente, reduciendo el uso de ancho de banda en un 50% respecto a PCM sin procesar.

Opciones FFmpeg usadas en AISAK:

- reconnect 1 → Reconectar automáticamente si la conexión se pierde
- reconnect_streamed 1 → Reconectar incluso en streams (importante para Spotify)
- reconnect_delay_max 5 → Esperar máximo 5 segundos antes de reconectar
- vn → No procesar video, solo audio (-v = video, n = no)
- acodec libopus → Usar codec Opus para audio (nativo de Discord)
- ab 128k → Bitrate de audio: 128 kbps (balance calidad/ancho de banda)
- ac 2 → Audio channels: 2 (estéreo)
- ar 48000 → Audio sample rate: 48000 Hz (estándar Discord)
- f s16le → Formato de salida: PCM signed 16-bit little-endian
- → Output a stdout (pipe directo a discord.py)

6.4 Ejemplo: Reproducción con FFmpeg en discord.py

```
import discord from discord.ext import commands async def play_audio(voice_client, url): '''
Reproduce audio desde URL usando FFmpeg FFmpeg decodifica el stream y lo transmite a Discord
''' # Opciones FFmpeg para máxima compatibilidad ffmpeg_options = { 'before_options':
'-reconnect 1 -reconnect_streamed 1 -reconnect_delay_max 5', 'options': '-vn -acodec libopus
-ab 128k -ac 2 -ar 48000 -f s16le' } # Crear objeto de audio con FFmpeg audio_source =
discord.FFmpegPCMAudio( url, executable='ffmpeg', # Debe estar en PATH o especificar ruta
absoluta **ffmpeg_options ) # Reproducir en el canal de voz def after_playing(error):
'''Callback cuando termina la reproducción''' if error: print(f"Error durante reproducción:
{error}") else: print("Reproducción completada, pasando a siguiente canción")
voice_client.play( audio_source, after=after_playing # Ejecutar después de terminar ) #
Esperar a que termine while voice_client.is_playing(): await asyncio.sleep(1)
```

7. HOSTING EN HUGGING FACE SPACES - GUÍA COMPLETA

7.1 ¿Por qué Hugging Face Spaces?

Hugging Face Spaces es la plataforma elegida para AISAK por múltiples razones: ✓ **Completamente Gratuito:** Sin límites de tiempo, sin tarjeta de crédito requerida, sin costos ocultos. A diferencia de Heroku (antes gratuito, ahora requiere pago) o AWS Lambda (con límites estrictos), HF Spaces es verdaderamente libre para uso indefinido.

✓ **Soporte Docker Nativo:** Permite especificar exactamente qué dependencias (FFmpeg, libopus, etc.) se instalarán. Mucho mejor que plataformas que solo soportan Python sin control sobre sistema operativo.

✓ **Persistent Storage (Opcional):** Si el bot necesita cachés o logs, pueden almacenarse persistentemente.

✓ **Escalabilidad:** El contenedor se ejecuta en hardware de Google Cloud, garantizando que múltiples servidores Discord pueden usar el bot simultáneamente sin degeneración de rendimiento.

✓ **Integración Comunitaria:** Repositorio público visible a la comunidad, facilita colaboración y obtención de feedback. El code es abierto y reproducible.

✓ **Variables de Entorno Seguras:** HF Spaces proporciona sección "Secrets" para guardar DISCORD_TOKEN de forma segura, sin exponerlas en el repositorio.

7.2 Dockerfile Optimizado para AISAK

El Dockerfile es el "plano" que describe cómo construir la imagen Docker. Se ejecuta una sola vez cuando se crea el Space, y crea un contenedor listo para ejecutar el bot:

```
# Dockerfile - Especificación de imagen Docker para AISAK # Usar imagen base oficial de
Python slim (ligera) FROM python:3.11-slim # Establecer variables de entorno para Python ENV
PYTHONUNBUFFERED=1 ENV PIP_NO_CACHE_DIR=1 # Actualizar índices de paquetes e instalar
dependencias del sistema RUN apt-get update && apt-get install -y --no-install-recommends \
ffmpeg \ # Procesador de audio (CRÍTICO) libopus0 \ # Codec de voz (CRÍTICO para Discord)
libsodium23 \ # Criptografía (necesario para discord.py) git \ # Git (opcional, para clonar
durante build) && rm -rf /var/lib/apt/lists/* # Limpiar caché para reducir tamaño de imagen #
Crear directorio de trabajo WORKDIR /app # Copiar requirements.txt (NO el código aún, por
caching de capas) COPY requirements.txt . # Instalar dependencias Python RUN pip install
--upgrade pip && \ pip install -r requirements.txt # AHORA copiar el código (esto permite
reutilizar capas anteriores si requirements.txt no cambió) COPY . . # Verificar que ffmpeg
esté disponible RUN which ffmpeg && ffmpeg -version | head -1 # Comando por defecto: ejecutar
el bot CMD ["python", "main.py"] # Healthcheck (opcional, pero recomendado) HEALTHCHECK
--interval=30s --timeout=10s --start-period=40s --retries=3 \ CMD curl -f
http://localhost:8080/health || exit 1
```

7.3 Proceso Paso a Paso: Crear y Configurar el Space

PASO 1: Crear nuevo Space en Hugging Face

- 1.1 Ir a huggingface.co y loguear (crear cuenta si no tiene)
- 1.2 Navegar a <https://huggingface.co/spaces>
- 1.3 Click "Create new space"
- 1.4 Nombre: "aisak-bot" (o nombre deseado)

- 1.5 Descripción: "Advanced AI-powered music bot for Discord"
- 1.6 Seleccionar Space type: **Docker** (IMPORTANTE)
- 1.7 Seleccionar runtime: Docker
- 1.8 Click "Create space"

PASO 2: Subir código

- 2.1 Clone el espacio localmente: `git clone https://huggingface.co/spaces/tu-usuario/aisak-bot`
- 2.2 Copiar archivos:
 - main.py
 - config.py
 - keep_alive.py
 - requirements.txt
 - Dockerfile
 - .gitignore (incluir .env)
 - carpetas: cogs/, utils/
- 2.3 `git add .`
- 2.4 `git commit -m "Initial bot setup"`
- 2.5 `git push`

PASO 3: Configurar Secrets (Credenciales Seguras)

- 3.1 En la página del Space, ir a Settings > Repository secrets
- 3.2 Click "Add a new secret"
- 3.3 Name: DISCORD_TOKEN
- 3.4 Value: (pegar el token de Discord)
- 3.5 Agregar otros secrets si es necesario (SPOTIFY_CLIENT_ID, etc.)
- 3.6 Click "Save"

PASO 4: Esperar a que se construya la imagen

- 4.1 Hugging Face iniciará automáticamente la construcción del Docker
- 4.2 Ir a tab "Logs" para ver progreso
- 4.3 Esperar a que diga "Building passed" (puede tardar 5-10 minutos)
- 4.4 Una vez completado, el bot debería conectarse a Discord automáticamente

PASO 5: Verificar que el bot esté en línea

- 5.1 Ir a tu servidor Discord
- 5.2 Debería ver el bot en la lista de usuarios
- 5.3 El status debe decir "Listening to /help para más comandos"
- 5.4 Probar comando: /help
- 5.5 Si funciona, ¡el bot está vivo!

7.4 Troubleshooting: Errores Comunes en HF Spaces

■ "Build failed: FFmpeg not found"

→ Solución: Verificar que Dockerfile incluye: `RUN apt-get install ffmpeg`

■ "ModuleNotFoundError: No module named 'discord'"

→ Solución: Verificar requirements.txt incluye `discord.py==2.3.2`

■ "Bot no se conecta a Discord"

→ Solución: Ir a Settings > Repository secrets y verificar DISCORD_TOKEN está configurado

■ **"Container keeps restarting"**

→ Solución: Ver logs detallados en tab "Logs" del Space. Buscar línea "Error" para diagnóstico.

■ **"FFmpeg version too old"**

→ Solución: En Dockerfile, reemplazar "python:3.11-slim" con imagen más reciente, ej: "python:3.12"

■ **"Connection refused to Discord"**

→ Solución: Verificar que el token es válido. Revisar en Discord Developer Portal que el bot está creado.

8. CONFIGURACIÓN DE UPTIMEROBOT - GUÍA DETALLADA

8.1 El Problema: Hibernación de Hugging Face Spaces

¿Qué es hibernación en HF Spaces?

Hugging Face Spaces está diseñado para ser de bajo costo. Por ello, si un Space no recibe tráfico HTTP durante 48 horas, el contenedor se "hiberna" (pausa). Esto significa:

Antes de hibernación:

- El bot responde a comandos Discord: ✓ Funciona
- El servidor Flask responde a peticiones HTTP: ✓ Funciona
- El bot está "listening" a eventos de Discord: ✓ Conectado

Después de hibernación:

- El contenedor Docker se pausa
- El bot no puede recibir eventos de Discord: ✗ Inactivo
- Los usuarios ven el bot "offline": ✗ No funciona
- El servidor Flask no responde: ✗ Inaccesible

¿Cómo evitar hibernación?

Hacer peticiones HTTP al endpoint Flask cada menos de 48 horas. Esto "despierta" el contenedor y lo mantiene activo indefinidamente. Este es exactamente lo que hace UptimeRobot.

8.2 Paso a Paso: Configurar UptimeRobot

PASO 1: Crear cuenta en UptimeRobot

- 1.1 Ir a <https://uptimerobot.com>
- 1.2 Click "Sign Up Free"
- 1.3 Ingresar email, crear contraseña
- 1.4 Confirmar email (revisar bandeja de spam)
- 1.5 Loguear a la cuenta

PASO 2: Obtener URL del Space

- 2.1 Ir al Space AISAK en Hugging Face
- 2.2 Copiar URL: <https://tu-usuario-huggingface.hf.space/>
- 2.3 Nota: URL debe terminar en .hf.space (sin /api)

PASO 3: Crear Monitor en UptimeRobot

- 3.1 En dashboard de UptimeRobot, click "Add New Monitor"
- 3.2 Llenar formulario:
 - Monitor Type: HTTP(s) (seleccionar)
 - Friendly Name: "AISAK Bot" (nombre descriptivo)
 - URL: <https://tu-usuario-huggingface.hf.space/>
 - Monitor Interval: 5 minutes (cada 5 minutos) - IMPORTANTE
 - HTTP Method: GET (dejar por defecto)
- 3.3 Click "Create Monitor"

PASO 4: Verificar que funciona

- 4.1 En la lista de monitores, debería ver "AISAK Bot" con status verde

- 4.2 Click en el monitor para ver detalles
- 4.3 Debería mostrar "Status: Up" y "Last check: [tiempo reciente]"
- 4.4 Si muestra rojo o "Down", revisar URL o que el Space esté corriendo

PASO 5: (Opcional) Configurar Notificaciones

- 5.1 En el monitor, click "Edit"
- 5.2 Bajar a sección "Notifications"
- 5.3 Agregar correo de notificación si está "Down"
- 5.4 Click "Save"

RESULTADO FINAL:

UptimeRobot hace GET a <https://tu-usuario-huggingface.hf.space/> cada 5 minutos.
Flask responde "AISAK Bot está en línea" con HTTP 200.
HF Spaces ve que hay tráfico y NO hiberna el contenedor.
Bot permanece online indefinidamente.

8.3 Monitoreo y Alertas

Dashboard de UptimeRobot:

- **Response Time:** Debería ser ~100-500ms. Si es mayor, investigar performance del Space.
- **Last Check:** Debería ser hace menos de 5 minutos. Si es más, UptimeRobot tiene problemas.
- **Uptime %:** Debería ser 99%+ después de varias semanas.
- **Status History:** Gráfico que muestra disponibilidad en el tiempo.

Si el monitor muestra "Down":

1. Verificar que el Space está corriendo (ir a HF Spaces, revisar logs)
2. Verificar que Flask está ejecutándose (en logs de HF debe verse "Serving Flask app")
3. Verificar que la URL es correcta (copiar/pegar nuevamente de HF)
4. Esperar 5 minutos y refrescar UptimeRobot (a veces hay delay en propagación DNS)

9. VARIABLES DE ENTORNO Y CONFIGURACIÓN SEGURA

9.1 Archivo .env Detallado

El archivo .env contiene credenciales sensibles y configuración específica del entorno. NUNCA debe ser committeado a git. Incluir .env en .gitignore:

```
# .env - Variables de Entorno (NO COMMIT A GIT) # Este archivo debe estar en .gitignore #
===== # CREDENCIALES CRÍTICAS #
===== # Token único del bot Discord (obtenido de
Developer Portal) # Nunca compartir públicamente. Si se expone, resetear inmediatamente en
Discord DISCORD_TOKEN=MTE1NDk5MTYzODk1MTA4MzIwMA.GabcDe.AbCdEfGhIjKlMnOpQrStUvWxYzAbCdEfGhIj
# ===== # INTEGRACIONES OPCIONALES #
===== # Spotify API (opcional, para búsqueda
mejorada) SPOTIFY_CLIENT_ID=4a3c5b7c9e1f3d5a7b9c1d3e5f7a9b1c
SPOTIFY_CLIENT_SECRET=9f8e7d6c5b4a3f2e1d0c9b8a7f6e5d4c # Last.fm API (opcional, para
información de canciones) LASTFM_API_KEY=abc1234567890def1234567890abcdef #
===== # CONFIGURACIÓN DEL SERVIDOR #
===== # Puerto en el que corre Flask (HF Spaces usa
8080 por defecto) FLASK_PORT=8080 # Host en el que escucha Flask (0.0.0.0 permite conexiones
desde cualquier IP) FLASK_HOST=0.0.0.0 # ===== #
LOGGING Y DEBUGGING # ===== # Nivel de log: DEBUG,
INFO, WARNING, ERROR, CRITICAL LOG_LEVEL=INFO # Guardar logs en archivo (ruta)
LOG_FILE=./logs/bot.log # ===== # CONFIGURACIÓN DEL
BOT # ===== # Prefijo para comandos tradicionales (si
se usan, además de slash commands) BOT_PREFIX=! # Color para embeds del bot (en formato
hexadecimal) BOT_COLOR=0x1f77b4 # Timezone para timestamps (ej: America/Santo_Domingo)
TIMEZONE=UTC # ===== # OPCIONES DE REPRODUCCIÓN #
===== # Volumen por defecto (0-100) DEFAULT_VOLUME=70
# Máximo de canciones en cola por usuario (anti-spam) MAX_QUEUE_LENGTH=100 # Timeout de
inactividad en canal de voz (segundos) INACTIVITY_TIMEOUT=300
```

9.2 Cómo Obtener el Discord Token

PASO 1: Acceder a Discord Developer Portal

- 1.1 Ir a <https://discord.com/developers/applications>
- 1.2 Loguear con la cuenta Discord que administra tu servidor

PASO 2: Crear Nueva Aplicación

- 2.1 Click "New Application"
- 2.2 Nombre: "AISAK"
- 2.3 Aceptar términos
- 2.4 Click "Create"

PASO 3: Obtener el Token

- 3.1 En el panel izquierdo, click "Bot"
- 3.2 Click "Add Bot"
- 3.3 Bajo "TOKEN", click "Copy"
- 3.4 Pegar en DISCORD_TOKEN en archivo .env (local) o en Secrets de HF (producción)

PASO 4: Configurar Permisos del Bot

- 4.1 En panel izquierdo, click "OAuth2" → "URL Generator"
- 4.2 Seleccionar scopes: bot

4.3 Seleccionar permisos:

- ✓ Send Messages
- ✓ Connect (Voice)
- ✓ Speak (Voice)
- ✓ Manage Messages
- ✓ Embed Links
- ✓ Attach Files

4.4 Copiar URL generada

PASO 5: Invitar Bot al Servidor

5.1 Pegar URL en navegador

5.2 Seleccionar servidor donde agregar el bot

5.3 Click "Authorize"

5.4 Resolver captcha

5.5 Ver confirmación "Bot joined the server"

IMPORTANTE - SEGURIDAD:

- NUNCA compartir el token públicamente
- Si el token se expone, ir a Discord Developer Portal y hacer click "Regenerate" inmediatamente
- Mantener tokens en variables de entorno, nunca en código fuente

10. CONSIDERACIONES AVANZADAS DE SEGURIDAD

1. Gestión Segura de Credenciales

- **Almacenamiento:** DISCORD_TOKEN solo en variables de entorno, nunca en .py files
- **Rotación:** Si token se expone, regenerar inmediatamente en Discord Developer Portal
- **Acceso:** En HF Spaces, usar sección "Secrets" que encripta valores en reposo
- **Auditoría:** No guardar tokens en logs. Si necesitas debuggear, usar masked tokens ej: "MTE...XYZ"

2. Rate Limiting y Prevención de Abuso

- **Por Usuario:** Máximo X comandos /play por minuto (evita spam de búsqueda)
- **Por Servidor:** Máximo Y conexiones simultáneas al bot (evita DoS)
- **Por Canción:** Timeout de Z segundos para reproducir (evita requests infinitas)
- **Implementar:** Usar discord.ext.tasks para contadores que se resetean periódicamente

3. Validación de Entrada

- Validar que URLs son URLs válidas (no ejecutar SQL injection)
- Validar longitud de query (máximo 500 caracteres)
- Sanitizar caracteres especiales en nombre de canción
- Usar try-except en todas las llamadas a yt-dlp

4. Manejo de Errores sin Exposición de Información

- INCORRECTO: await ctx.send(f"Error: {full_exception_message}")
- CORRECTO: await ctx.send("■ Error al reproducir canción. Por favor intenta de nuevo.")
- Razón: Errores detallados pueden revelar información de infraestructura
- Guardar detalles en logs privados, no en chat público

5. Permisos Mínimos del Bot

Discord permite asignar permisos granulares. AISAK solo necesita:

- Send Messages (comunicar estado)
- Connect to Voice (unirse a canales de voz)
- Speak (reproducir audio)
- Manage Messages (limpiar comandos antiguos, opcional)
- NO necesita: Administrator, Ban Members, Kick Members, etc.

6. Logging Seguro

- Log: "Usuario X solicitó canción en servidor Y" (información pública)
- NO log: "Discord token es XYZ123..." (información privada)
- NO log: Contraseñas de API, credenciales sensibles
- Usar logging.Logger con niveles: DEBUG (dev), INFO (producción), ERROR (problemas)

7. Comunicación HTTPS

- Todas las peticiones HTTP a APIs (YouTube, Spotify, etc.) deben ser HTTPS
- Verificar certificados SSL/TLS
- No hacer peticiones HTTP a endpoints no confiables
- En yt-dlp, validar URLs descargadas de sitios públicos

11. MANEJO DE ERRORES Y DEBUGGING

11.1 Errores Comunes en Producción

ESCENARIO 1: "Bot offline después de 48 horas"

Síntomas: Bot funciona un tiempo, luego desaparece de la lista de usuarios

Causa: HF Spaces hibernó el contenedor (inactividad > 48 horas)

Solución: Configurar UptimeRobot como se describe en Sección 8

Prevención: Monitoreado automáticamente después de UptimeRobot está activo

ESCENARIO 2: "FFmpeg no encontrado" en logs

Síntomas: Comando /play devuelve "Error procesando audio"

Causa: FFmpeg no instalado en el contenedor Docker

Solución: En Dockerfile, agregar: RUN apt-get install ffmpeg

Depuración: En logs de HF Spaces, buscar "FileNotFoundError: ffmpeg"

ESCENARIO 3: "No reproduce audio, solo silencio"

Síntomas: Bot conecta al canal, música no se escucha

Causa: Múltiples posibles: FFmpeg falla, yt-dlp no extrae URL, permisos de voz insuficientes

Solución: Revisar logs de HF Spaces línea por línea. Buscar stack trace.

Depuración: Agregar más print/logging en función play() para diagnosticar en qué paso falla

ESCENARIO 4: "Discord token inválido"

Síntomas: Bot no conecta, error "Invalid token" en logs

Causa: Token expirado, incorrecto, o regenerado

Solución: Ir a Discord Developer Portal, regenerar token, actualizar en HF Secrets

Prevención: No regenerar tokens sin motivo. Si alguien ve el token, ENTONCES regenerar.

ESCENARIO 5: "yt-dlp falla para sitios específicos"

Síntomas: Comando /play funciona para YouTube pero no para Spotify

Causa: Sitio cambió estructura, yt-dlp está desactualizado

Solución: Actualizar yt-dlp a última versión en requirements.txt

Depuración: Crear issue en github.com/yt-dlp/yt-dlp si persiste

ESCENARIO 6: "Bot responde lentamente a comandos"

Síntomas: Hay delay entre escribir /play y que responda

Causa: HF Space está sobrecargado, yt-dlp tardando, internet lenta

Solución: Implementar async/await correctamente. Usar aiohttp en lugar de requests.

Optimización: Cachear resultados de búsqueda recientes (últimas 100 búsquedas)

ESCENARIO 7: "Permisos insuficientes" en error

Síntomas: Comando /play devuelve "Missing permissions"

Causa: Bot no tiene permisos CONNECT o SPEAK en el canal de voz

Solución: Ir a Settings → Roles del servidor Discord, asignar permisos a rol del bot

Verificación: El rol del bot debe estar ARRIBA en jerarquía de otros roles regulares

11.2 Herramientas de Debugging

1. Logs de HF Spaces

Location: Space página principal → tab "Logs"

Información: Salida estándar (stdout) + errores estándar (stderr)

Cómo leer: Buscar líneas con "ERROR", "Traceback", "Exception"

Límite: Últimas 100 líneas visible en UI, pero completo en CLI via `huggingface-hub`

2. Discord Developer Portal

Location: discord.com/developers/applications

Información: OAuth2 logs, permisos del bot, webhooks

Útil para: Verificar que bot se conectó, revisar qué permisos tiene

3. Logging en Python

```
```python import logging logger = logging.getLogger(__name__) # En tu código try: info = await
extract_audio(query) logger.info(f"✓ Audio extraído: {info['title']}") except Exception as e: logger.error(f"✗ Error
extrayendo: {str(e)}") ```
```

### 4. Testing Local Antes de Deploy

Ambiente: Instalar Python 3.11, pip install -r requirements.txt

Testing: Crear bot test en Discord Developer Portal

Ejecutar: python main.py localmente

Ventaja: Ver logs en tiempo real en tu terminal, debuggear paso a paso

## 12. INFORMACIÓN REQUERIDA PARA CONEXIÓN A DISCORD

### 12.1 Checklist Pre-Conexión

Antes de conectar el bot a Discord, el agente verificará que tienes:

Item	Descripción	Dónde Obtener
Discord Token	Credencial única del bot	Discord Developer Portal > Applications > Bot > Copy Token
Application ID	ID único de la aplicación	Discord Developer Portal > Applications > General Information
Server ID (opcional)	ID del servidor para testing	Click derecho en servidor Discord > Copy ID (activar Dev Mode)
Channel ID (opcional)	ID del canal para testing	Click derecho en canal Discord > Copy ID
Account Email	Email de tu cuenta Discord	Tu cuenta Discord

### 12.2 Flujo Final de Conexión y Validación

#### FASE 1: RECOPIACIÓN DE CREDENCIALES

El agente te pedirá:

1. "¿Cuál es tu Discord Bot Token?" → Tú proporcionas token de Developer Portal
2. "¿Cuál es tu Application ID?" → Tú proporcionas ID de la aplicación
3. "¿Tienes un servidor de testing?" → Tú proporcionas Server ID (opcional pero recomendado)

#### FASE 2: GENERACIÓN DE LINK DE INVITACIÓN

El agente generará un link OAuth2 con los permisos necesarios:

[https://discord.com/api/oauth2/authorize?client\\_id=\[TU\\_APP\\_ID\]&permissions=36554816&scope=bot](https://discord.com/api/oauth2/authorize?client_id=[TU_APP_ID]&permissions=36554816&scope=bot)  
(Permisos: Send Messages, Connect, Speak, Manage Messages)

#### FASE 3: INVITACIÓN DEL BOT AL SERVIDOR

Tú haces clic en el link, seleccionas servidor, autorizas.

Discord te devuelve confirmación: "Bot joined the server"

Verificación: En tu servidor Discord, debería aparecer el bot en la lista de usuarios.

#### FASE 4: VALIDACIÓN INICIAL

El agente emitirá comando /help para verificar que el bot responde.

Debería responder con embed mostrando todos los comandos disponibles.

Si responde: ✓ Bot está en línea y funcional

Si no responde: Revisar que token es correcto en HF Secrets

#### FASE 5: PRUEBAS DE FUNCIONALIDAD

Agente ejecutará secuencia de pruebas:

1. /play "Test Song" → Debería conectar a canal de voz y reproducir
2. /pause → Debería pausar sin errors
3. /resume → Debería reanudar desde donde pausó
4. /skip → Debería reproducir siguiente

- 5. /queue → Debería mostrar próximas canciones
- 6. /stop → Debería desconectar del canal de voz

## FASE 6: PRODUCTION READY

Una vez que todos los comandos funcionan: • Bot está online 24/7

- UptimeRobot mantiene HF Spaces despierto
- Flask responde a health checks
- Discord está recibiendo eventos correctamente
- Bot está listo para uso en producción

## SIGUIENTES PASOS:

- Invitar más usuarios a probar el bot
- Recopilar feedback para mejoras
- Monitorear logs en HF Spaces para bugs
- Actualizar dependencias mensualmente

## 12.3 Lo Que Sucede Detrás de Escenas

Cuando presionas "Authorize" en el link OAuth2, Discord realiza:

### 1. Validación de Credenciales

Discord verifica que Application ID y Token corresponden a la misma aplicación

### 2. Verificación de Permisos

Discord verifica que la aplicación solicitó permisos válidos

### 3. Creación de Guild Member

Discord agrega un "miembro" bot a tu servidor, asignándole el rol @aisak-bot

### 4. Inicialización de Websocket

El bot establece conexión WebSocket con Gateway de Discord  
Discord comienza a enviar todos los eventos del servidor al bot  
Bot puede ahora escuchar mensajes, cambios de voz, etc.

### 5. Sincronización de Slash Commands

Cuando el bot se conecta, discord.py llama bot.tree.sync()  
Esto registra todos los @app\_commands decorados en Discord  
Los usuarios ahora ven /play, /pause, /skip, etc. en el autocomplete

### 6. Estado Inicial

El bot carga en la lista de usuarios del servidor  
Presencia se establece: "Listening to /help para más comandos"  
Bot está listo para recibir comandos





*Documento Técnico Completo del Bot AISAK | Generado: 19 de April de 2026 a las 21:58:06 | Versión 2.0 Expandida | Esta especificación es exhaustiva y proporciona todos los detalles necesarios para desarrollar un bot Discord de producción. Sigue este documento paso a paso para garantizar un resultado profesional y robusto.*